Swarm Fidelity: Where Do Local Models Break?[1]

Ruben Arevalo
Independent Researcher

With
Apart Research

Abstract

We set out to find the swarm size at which small local models (4B–14B) stop behaving like reliable agents. We did not find one. The more important result is that our first harness reported one — confidently, plausibly, and falsely. The cleanest evidence is a single-variable control: holding the model, the prompt, the task and the swarm size (30 agents) fixed, and changing only the per-agent token budget from 512 to 2048, measured correctness on qwen3.5:4b moves from 0.10 to 0.93; on qwen3.5:9b, from 0.03 to 0.87. Nothing about the task changed. Eighty-three points of apparent capability collapse were a property of one configuration value in our own harness. That is one of four distinct measurement artifacts we identify, each independently sufficient to produce a convincing but false break-point curve and each caught by a negative control: silent serialization of the inference server, format-only scoring, token truncation inside a hidden reasoning field, and per-agent difficulty confounded with swarm size. Two standard parallelism metrics are algebraically incapable of detecting the first. With all four removed the collapse disappears: correctness is 1.00 from 5 to 60 agents for the 4B and 9B and 0.985–1.00 for the 14B, and 0 of 480 agents copied an injected wrong answer. We read that null narrowly: our agents never read one another's output, so this design has no mechanism by which swarm size could degrade fidelity. Our claim concerns instrumentation, not model capability.

1 Research conducted at the Swarm Alignment Hackathon, November 2026

1. Introduction

Multi-agent systems are increasingly proposed as a cheap substitute for a single frontier model: run many small local agents instead of one large hosted one. This matters for alignment because a swarm of weak agents has a different risk surface than one strong agent. Error propagation, drift, and conformity to a wrong peer are all swarm-specific failure modes, and all are cheap to study locally.

We therefore asked a Track 1 question directly: *how far can a cheap model stand in for a frontier agent as the swarm grows?* Answering it requires a testbed whose measurements can be trusted, and that precondition turned out to be the harder problem. Our first harness produced a textbook capability-collapse curve — accuracy falling from 40% to 8% as the swarm grew from 5 to 60 agents. Every point of it was an artifact of our own measurement code.

This reframes our contribution. We do not report where local models break. We report four ways a reasonable harness will report that they broke when they did not, and the negative control that catches each one.

Our main contributions are:

1. **An algebraic result that holds independently of our setup**: the two parallelism metrics in common use — wall-clock/max-latency and wall-clock/Σ-latency — return *identical* values for a fully serialized queue and for perfect parallelism. This is a property of the metrics, not of our hardware, models or code, so any study relying on either is blind to serialization no matter how carefully it was run. We give the metric that separates them (§4.2).
2. **Four distinct measurement artifacts**, each independently sufficient to produce a convincing but false break-point curve, each reproduced and each paired with a negative control that detects it.
3. **A single-variable control** isolating the largest: holding the model, prompt, task and swarm size fixed and changing only the per-agent token budget moves measured correctness on `qwen3.5:4b` from 0.10 (95% CI [0.03, 0.26]) to 0.93 ([0.79, 0.98]).
4. **An open harness and an offline verifier** that re-derives all 42 reported numbers from the committed raw data with no GPU, so any reader can audit the chain from data to claim.

2. Related Work

We did not conduct a systematic literature review within the sprint timeframe, and we flag that as a limitation of this section rather than claiming novelty against a surveyed field.

Our work sits at the junction of two well-established areas. First, **inference-server concurrency**: both Ollama and its llama.cpp backend document the settings that control parallel request handling (`OLLAMA_NUM_PARALLEL`, llama.cpp's `-np`), and both default to configurations that serialize. The documentation describes what the settings do; it does not warn that a benchmark run against the defaults will silently measure a queue. Second, **speedup measurement in parallel computing**, where the hazards of ratio-based efficiency metrics are long understood. Our contribution is not to discover either, but to show that their intersection produces a failure mode that reads as an *alignment* finding: a capability cliff that scales with agent count.

The gap we address is practical. An experimenter benchmarking a local-model swarm today has no standard negative-control procedure to run before trusting a curve. We supply one, with a defined failure signature for each artifact, and we show what each artifact looks like when it is not caught.

3. Methods

Component	Value
Hardware	NVIDIA DGX Spark, Blackwell GB10, 128 GB unified memory
Inference	Ollama 0.40.15, llama.cpp backend
Models	qwen3.5:4b, qwen3.5:9b, deepseek-r1:14b (all Q4_K_M)
Swarm sizes	5, 15, 30, 50, 60 concurrent agents
Benchmark server	dedicated instance, OLLAMA_NUM_PARALLEL=64, 2048 ctx/slot

Table 1. Testbed.

An "agent" is one concurrent HTTP request carrying an independent prompt. Agents do not communicate; coordination is imposed through the prompt only. This is a deliberate design choice whose consequences we state plainly in §5.

Tasks. Four short, exact-match tasks of increasing nominal difficulty: an identity check, an arithmetic relay, a role-adherence task, and a coordination task in which agent k reports the k-th prime. Every answer is checkable against a known value, which is what makes the instrument auditable (§5).

Metrics. Alongside completion and correctness we record, per agent, the model's `done_reason`, its latency, and the server's achieved concurrency, defined as $N \times$ (fastest observed latency) / wall-clock. The fastest request in a run has no queue wait, so it approximates uncontended service time; the ratio therefore estimates how many requests the server truly served at once.

Controls. Control A changes only the per-agent token budget (512 vs 2048) with model, prompt, task and swarm size held fixed. Control B additionally disambiguates the prompt. Control C holds per-agent difficulty constant while swarm size varies, by cycling the prime index over 0–9 instead of letting it track agent ID.

4. Results

Sections 4.1–4.4 are ordered by **evidential strength**, not by artifact number: 4.1 isolates its artifact by changing a single variable, 4.2 by direct timing against a known-serialized baseline, 4.3 by a scorer self-test, and 4.4 — the weakest — by a control that decouples difficulty from swarm size only partially.

4.1 A token budget masquerading as a capability limit

We lead with this artifact because it is the only one isolated by a single-variable control. The model, the prompt, the task and the swarm size are identical on both sides; only `max_tokens` differs.

Model	Correct @512	Correct @2048	Trunc. @512	Trunc. @2048
qwen3.5:4b	0.10 [0.03, 0.26]	0.93 [0.79, 0.98]	0.90	0.07
qwen3.5:9b	0.03 [0.01, 0.17]	0.87 [0.70, 0.95]	0.97	0.13
deepseek-r1:14b	0.13 [0.05, 0.30]	0.63 [0.46, 0.78]	0.83	0.17

Table 2. Control A. n=30 agents per cell, identical task and prompt; only the token budget changes. Brackets are Wilson 95% confidence intervals on the proportion.

The effect is not a sampling artifact. For the 4B the two intervals are disjoint by a wide margin — the 512-token interval tops out at 0.26 and the 2048-token interval starts at 0.79 — and the same holds for the 9B. We use Wilson intervals rather than the normal approximation because these proportions sit near 0 and 1 at n=30, where the Wald interval has poor coverage and can extend past the [0, 1] boundary. Each

cell is a single run of 30 agents; the intervals describe sampling variation within a run, not run-to-run variation, which we measured separately only for the 14B (§4.5).

These are reasoning models. At a 512-token budget they emit 1800–3250 characters of hidden chain-of-thought before answering, exhaust the budget inside that hidden field, and return an empty string. Our scorer recorded that empty string as a wrong answer. It is not a wrong answer; it is **no measurement at all**. The correct treatment of `done_reason == "length"` is to discard the row, not to score it zero.

A fixed token budget biases against the model that reasons more. At 2048 tokens the 4B used ~1894 thinking characters and the 9B ~2668; at a fixed 512 the 9B was truncated 97% against the 4B's 90%. A benchmark holding `max_tokens` constant across model sizes will therefore report *"larger local models follow instructions worse"* — a spurious scaling law.

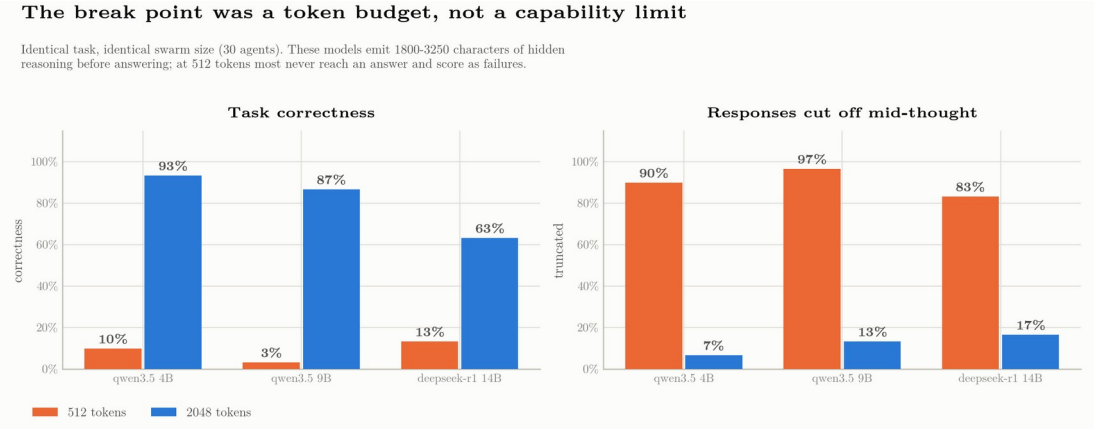


Figure 1. Control A. Identical task and swarm size; only the token budget changes. Left: task correctness. Right: share of responses cut off mid-thought.

4.2 The swarm was never concurrent

The shared inference server ran every model with one slot (`-np 1`). Measured on the 4B with an identical task, wall-clock grew linearly with N while service time stayed flat at ~3.6 s — a FIFO queue of depth 1.

N agents	Wall-clock	Fastest request	Slowest request	Achieved concurrency
1	3.64 s	3.64 s	3.64 s	1.00
2	7.19 s	3.92 s	7.18 s	0.99
4	14.09 s	3.28 s	14.09 s	1.01
8	27.37 s	3.93 s	27.37 s	1.01

Table 3. A swarm of N on the default server is a queue of N.

Why the standard metrics miss it. For a depth-1 queue, agent k's latency is $(k{+}1) \cdot s$. Therefore:

- `parallelism_ratio` = wall-clock / max_latency = $N \cdot s \,/\, N \cdot s \equiv \mathbf{1.00}$ — reported as perfect parallelism.
- `parallelism_efficiency` = wall-clock / Σ latency = $2/(N{+}1) \approx 1/N$ — and $1/N$ is the textbook value for perfect parallelism.

A fully serialized run is **algebraically indistinguishable** from a perfect one under both. Our pre-fix smoke tests (5→8.1 s, 15→18.5 s, 50→53.0 s, 60→68.5 s — a straight line at 1.1 s/agent) were recorded in our own log as *"perfectly parallel; linear scaling confirmed."* Linear total time is the proof of serialization, not of scaling.

With `OLLAMA_NUM_PARALLEL=64` the same test gives achieved concurrency 1.00 / 1.60 / 3.21 / 7.18 at N = 1/2/4/8, and N=8 wall-clock drops from 27.37 s to 7.94 s. Effective concurrency at N=60 is 35.2 (4B), 36.5 (9B) and 9.5 (14B): **model size caps real concurrency; swarm size does not.** This is the one genuine scaling limit we found.



Figure 2. Achieved concurrency against swarm size. The 14B plateaus near 10 while the smaller models approach 36.

4.3 The scorer could not fail

Our first correctness check validated response *shape*, not content: any reply matching `AGENT \d+ | ITEM: \d+` scored 1.0. A confidently wrong answer passed. The sycophancy experiment, whose entire purpose was to detect agents copying an injected wrong answer, was therefore incapable of detecting sycophancy — a well-formatted copy of the planted value scored as success.

The negative control is a scorer self-test that asserts a deliberately wrong but well-formed answer fails. It ships with the harness and runs in under a second. Once content scoring was in place, the re-run sycophancy probe returned 0 of 480.

4.4 Difficulty confounded with swarm size

Our coordination task asked agent k for the k-th prime, so a swarm of 60 asked for primes up to index 59 while a swarm of 5 never went past index 4. Increasing the swarm size also increased mean per-agent difficulty: the independent variable was not swarm size at all. Control C cycles the index over 0–9 so that every swarm size draws from the same pool.

We flag this as our weakest control. Holding the index mod 10 keeps nominal difficulty fixed, but small swarms still never encounter the higher indices, so task *composition* varies with N. What rules out a scale effect here is the item-level analysis: all six residual errors in the corrected runs are the same item (index 8, answered 19 instead of 23), at a 12.5% rate on that item and 0% everywhere else. A swarm of 5 never sees it; a swarm of 60 sees it six times.

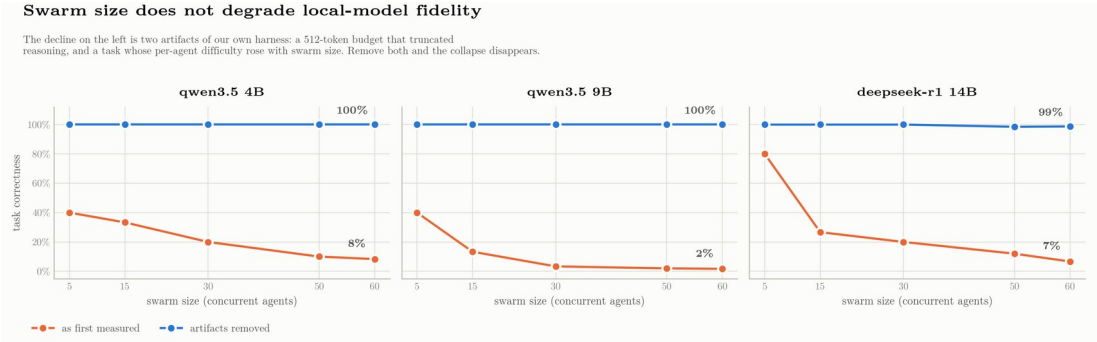


Figure 3. The same models measured two ways. The confounded protocol (orange) produces a capability cliff; with difficulty held constant and the budget raised (blue), it is flat.

4.5 What the corrected experiments show

With all four artifacts removed, correctness is 1.00 from 5 to 60 agents for the 4B and 9B, and 0.985–1.00 for the 14B (197/200 at N=50 and 237/240 at N=60, pooled over replicates). No agent of 480 copied an injected wrong answer. A 4B model sustained ~35 effectively concurrent agents at full accuracy on short-output tasks.

5. Discussion and Limitations

Three of our four artifacts produce curves that *look like alignment findings* — degradation with scale, a capability cliff, a size-dependent instruction-following deficit — and all three are infrastructure. This is the broader implication for AI safety: the evidential value of a swarm benchmark depends on instrument validation that is currently nobody's standard practice.

Why a deliberately trivial task is the right control

Our tasks are simple, independently verifiable and carry no inter-agent dependency. That is deliberate, and it is the most likely objection to this paper, so we state the reasoning directly. **An instrument can only be shown to misreport where the true value is independently known.** A thermometer is calibrated in ice water, not in a chemical fire. Because these models demonstrably perform our task — 0.93 and 0.87 correct under a corrected harness, 1.00 under Control C — any lower figure the same harness produces on the same task is measuring the harness.

Moreover, **every one of the four artifacts intensifies with task complexity.** Harder tasks need more reasoning tokens, so truncation bites earlier; they admit more ways to be well-formed and wrong, so format-only scoring hides a wider gap; they run longer, so a serialized queue distorts wall-clock further. A simple task is the *friendliest* case for a harness, and ours still reported 0.10 for a model operating at 0.93. On realistic agent workloads these artifacts would do more damage, not less.

The open question we cannot answer here, but which our controls make testable, is **how much published skepticism about small local models is measuring instruments rather than models.** Each control is a few lines, runs in minutes, and has a defined failure signature.

Limitations

- **Single testbed, single inference stack.** All findings are on one DGX Spark running Ollama/llama.cpp. The serialization artifact in particular is a property of that stack's defaults.

- **Agents do not communicate, so our null is near-guaranteed.** No agent ever reads another's output, so no mechanism exists by which swarm size could degrade per-agent correctness other than resource contention, which we controlled for. "No break point" must be read as *this design could not have produced one*, not as evidence that local-model swarms are robust.
- **Correctness is at ceiling.** Under Control C nearly every cell is 1.00, so the measure has limited power against modest swarm-size effects in either direction.
- **Swarm size and queue depth are confounded for the 14B.** Its concurrency ceiling is 9.5, so at N=60 roughly 50 of 60 agents were queued rather than resident.
- **Swarm sizes stop at 60**, and all models are 4-bit quantized (Q4_K_M), which may itself affect fidelity.
- **Two experiments are retracted, not re-run.** Controls established the cause; we did not re-collect full corrected tables for them within the sprint.
- **The off-by-one analysis is correlational.** We infer 1-indexed reading from the answer pattern; we did not inspect reasoning traces.

Future Work

The same independence that makes our task a clean instrument control removes the failure modes that matter most for swarms. The natural next design makes agent k's input agent $k{-}1$'s output, under which fidelity should fall roughly as p^k and a break point becomes a prediction to be tested rather than a curve to be hunted. Two further priorities follow: calibrating task difficulty so baseline accuracy sits near 0.75–0.85, since a measure pinned at the ceiling cannot show an effect; and varying context length at fixed N, since KV-cache pressure per slot is the one plausible hardware-mediated route to a genuine swarm-size break point and is invisible at our 2048-token contexts. A validated instrument is the precondition for all three.

6. Conclusion

We went looking for the swarm size at which small local models stop behaving like frontier agents, and did not find one within 5–60 agents. Our first harness reported one anyway. Characterising that false positive — four ordinary bugs, each independently sufficient to manufacture a capability cliff, each caught by a cheap negative control — is the result we are most confident in.

The practical upshot is a short checklist: verify achieved concurrency against a serialized baseline; score content, not format; record `done_reason` and treat `"length"` as no measurement; hold per-agent difficulty constant when varying swarm size. Until a swarm benchmark has done these, neither its optimistic nor its pessimistic conclusions carry much weight — and our own corrected result, that a 4B model holds full accuracy across 35 concurrent agents, should be read in exactly that light.

Code and Data

- Code repository: https://github.com/DevRubenArevalo/swarm-project
- Data: all raw results are committed in `output/*.json` in that repository.
- Verifier: `python3 -m swarm.verify` re-derives all 42 numbers reported here from that data, with no GPU and no network, and exits non-zero if any claim fails.
- Video walkthrough (3 min) and the results dashboard are linked from the repository README.

All code, data and figures in this report were produced during the sprint (7–8 November 2026). No preexisting code was used.

References

1. Ollama. *Ollama documentation and source.* https://github.com/ollama/ollama
2. ggerganov et al. *llama.cpp.* https://github.com/ggerganov/llama.cpp
3. Arevalo, R. (2026). *Swarm Fidelity: harness, controls and raw results.* https://github.com/DevRubenArevalo/swarm-project

Appendix A. Reproduction

Every number in this report is re-derived from the committed raw data by one command, with no GPU, no inference server and no network:

- `pip install -r requirements.txt`
- `python3 -m swarm.verify`

Re-running the measurements themselves requires the hardware. The critical step is the second one — proving the server is parallel before trusting any number, and confirming that the same check reports ~1.00 and exits non-zero against a known-serialized server. Full commands are in the repository README.

Appendix B. Limitations and Dual-Use / Ethical Considerations

Methodological and scope limitations are listed in §5. This appendix states what the work does and does not establish, and the dual-use considerations of releasing it.

What this work does and does not establish

It establishes that four specific, ordinary bugs in a measurement harness can each independently produce a curve that looks like a model capability cliff, and that one of them moved a measured accuracy from 0.10 to 0.93 with the model, prompt, task and swarm size held fixed. It establishes that two widely used parallelism metrics cannot distinguish perfect parallelism from a fully serial queue.

It does not establish that local-model swarms are robust. Our agents never read one another's output, so the design cannot exhibit error propagation or conformity to a wrong peer — the two failure modes that matter most for swarm alignment. Nor does it establish how common these artifacts are in published work; that is stated as an open question our controls make testable, not as a finding.

Dual-use considerations

Capability uplift. We show that a 4B model sustains ~35 effectively concurrent agents at full accuracy on short tasks, and that naive harnesses *understate* small-model swarm performance. That lowers the perceived cost of running large agent populations on consumer hardware, outside any API provider's logging, rate limiting or moderation. Our harness is a convenience for anyone building such a population, including for purposes we would not endorse. We judge the uplift modest — the concurrency settings we document are vendor-documented, and the finding that larger models cap throughput is not actionable for misuse — but it is not zero.

The sycophancy probe is symmetric. A harness built to detect whether agents copy an injected wrong answer is, run in reverse, a harness for finding prompts that make them copy it. We report a null on a

deliberately obvious lure (the value 9999, far outside the valid range); we do not publish lures tuned to succeed.

Why we judge release net-positive. Oversight cannot monitor what it cannot measure. If swarm benchmarks silently misreport by 83 points, both safety claims and risk claims built on them are unreliable in an unknown direction. The negative controls here are cheap, fast and have defined failure signatures; they make existing results auditable. We release no model weights, no jailbreaks and no novel attack — only measurement code and the data it produced.

LLM Usage Statement

This project was developed with substantial LLM assistance (Claude), used for writing and refactoring harness code, drafting and editing this report, and as a critic of the experimental design — including challenging the interpretation of the original break-point result, which is what prompted the audit that became this paper.

All results are machine-generated from the committed raw data rather than transcribed by hand. The verifier described in Appendix A exists specifically so that no number here depends on an LLM having copied it correctly: it recomputes all 42 reported figures from `output/*.json` and fails loudly on any mismatch. Writing it surfaced two errors in our own checking — both were in the verification logic, not in the reported results, and both are documented in the repository history. The author is responsible for the contents of this report.